

Developing a Secure, Backwards-Compatible ARP Protocol

What is the ARP protocol?

A means by which computers discover each other on a network. A device wanting to discover another will send out an ARP request and if all goes well, the correct device will send an ARP reply, letting the requester know its MAC address.

What is the issue with it?

There is no integrity verification of ARP replies. There are legitimate circumstances for a device to send an unsolicited ARP reply, such as sending gratuitous ARP replies following an IP address change, or after an NIC change. This will update the IP-MAC bindings in devices that receive the ARP reply. Since there is no verification of integrity, a device can update the IP-MAC bindings of a target device on the network so that the MAC address associated with the default gateway of the network is the attacker, allowing them to perform an MITM attack, in the worst case allowing credentials to be read and payloads injected into HTTP responses.



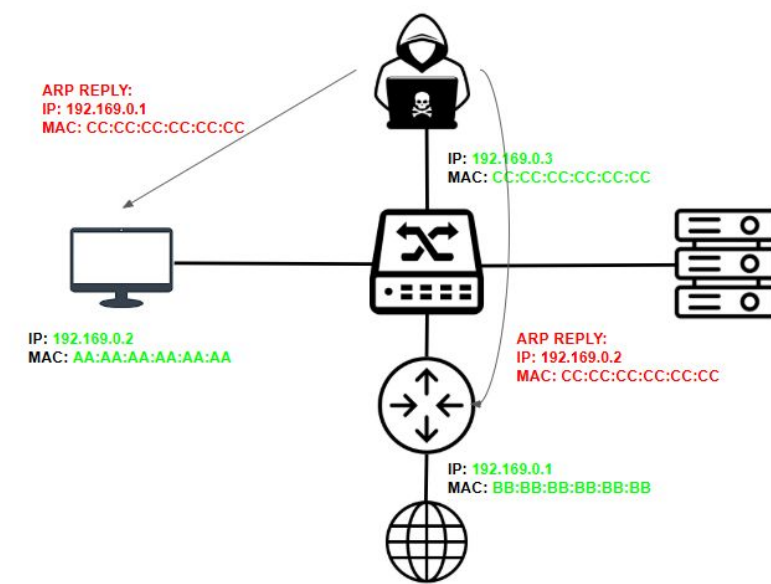
Link to ARP Spoofing Demo

A Decentralised Approach

Whenever a DHCP server leases an IP address to a device, it stores this IP-MAC binding and broadcasts this information to all switches on the network as a 'token', which will receive and store this lease information. This includes the time of the lease, and the expiry time is always an hour + time of lease. In the basic DHCP controller set up on the Mininet network, the DHCP server does not handle releases or renewals, however it would not be difficult to extend this concept to include this functionality. Whenever a client wishes to verify a IP-MAC binding in its table, it broadcasts a request for the relevant token, and this request will be handled by any switch that has the token, or the DHCP server. If no packets are lost, the switch on the client's subnet can respond with the token, putting less strain on the DHCP server.

Each token also contains a cryptographic signature that the client can use to verify the authenticity of the token. This is because, especially in wireless networks (even those secured with WPA2), it is possible for an attacker to forge packets and send them to the victim. On these wireless networks, using trusted and untrusted ports will not work either, since the attacker can send a packet directly to a victim without it being handled by the AP first as it would in standard 802.11 infrastructure mode. In WPA2, it is possible for an attacker to derive the PTK of a victim using the PSK and captured handshake. If a client receives no valid tokens, then it can assume an error in the IP-MAC binding. This is backwards compatible since a device without using this protocol will still be able to receive leases and populate its ARP table.

This has the disadvantages that each client on the network must have access to the RSA public key used for verifying the signatures of the tokens, and that the cryptographic process of creating and verifying these signatures has overhead.



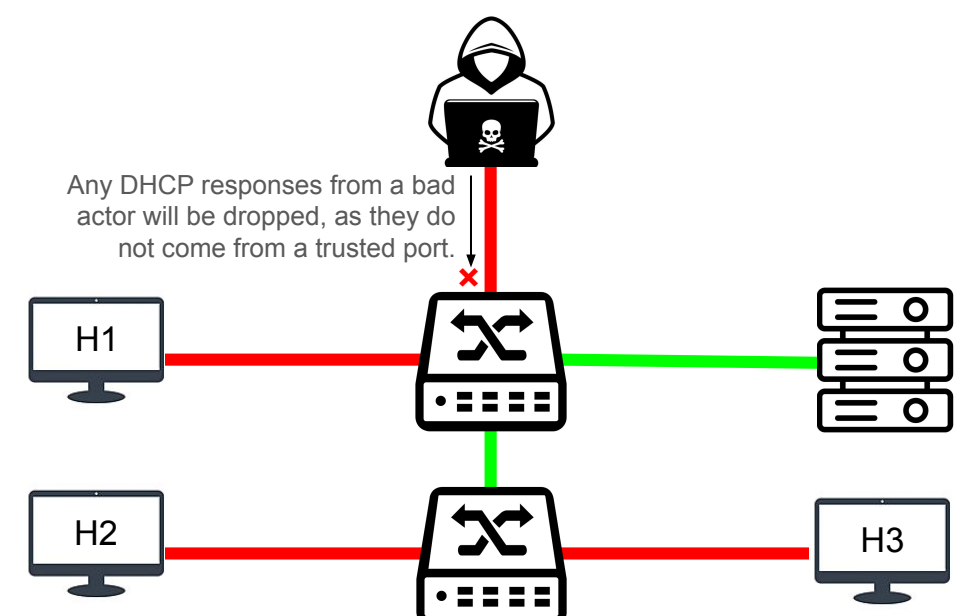
One possible solution is Dynamic ARP Inspection.

This works by each switch in the network storing a IP-MAC binding table, updated by DHCP server packets. If an ARP reply is sent from an **untrusted** port (e.g. not another switch, perhaps a client), the reply is verified using the switch's binding table, and dropped if the IP-MAC binding is not found.

A Centralised Approach

This solution involves querying the DHCP server for a MAC-IP address binding. Since the server must be contacted in order for a node to be assigned an IP address, the address of the DHCP server is known. This provides the benefit of unicast communication, occupying less bandwidth than a broadcast ARP request. Furthermore, this solution prevents the need for switches to dynamically inspect packets, simplifying the switches' role in the network.

This method remains vulnerable to DHCP spoofing, where a bad actor responds to broadcast DHCP requests. The DHCP protocol has no inherent safeguards against such an attack - the first received DHCP response at the node is assumed to be legitimate. A simple solution involves marking ports on switches as "trusted" or "untrusted": ports connecting switches to other switches or the DHCP server are marked as trusted, with all other ports marked as untrusted. DHCP responses are dropped when coming from an untrusted port.



This approach has the disadvantage of burdening the DHCP server with additional complexity. Additionally, it reinforces a single point of failure in the network. Now when the DHCP server goes down, not only does it prevent any new connections or IP re-bindings, but additionally prevents different devices from discovering each other.